Rails Is Agile

The title of this book is *Agile Web Development with Rails 7*. You may be surprised to discover that we don't have explicit sections on applying agile practices *X*, *Y*, and *Z* to Rails coding. In fact, you won't find mention of many agile practices, such as Scrum or Extreme Programming, at all.

Over the years since Rails was introduced, the term *agile* has gone from being relatively unknown to being overhyped, to being treated as a formal set of practices, to receiving a well-deserved amount of pushback against formal practices that were never meant to be treated as gospel, to a return back to the original principles.

But it's more than that. The reason is both simple and subtle. Agility is part of the fabric of Rails.

Let's look at the values expressed in the Agile Manifesto (Dave Thomas was one of the seventeen authors of this document) as a set of four preferences:[2]

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Rails is all about individuals and interactions. It involves no heavy toolsets, no complex configurations, and no elaborate processes, just small groups of developers, their favorite editors, and chunks of Ruby code. This leads to transparency; what the developers do is reflected immediately in what the customer sees. It's an intrinsically interactive process.

The Rails development process isn't driven by documents. You won't find 500-page specifications at the heart of a Rails project. Instead, you'll find a group of users and developers jointly exploring their need and the possible ways of answering that need. You'll find solutions that change as both the developers and the users become more experienced with the problems they're trying to solve.